ASSIGNMENT_CO2

Jithesh C(192524320)

Github link: https://github.com/jithesh192524320/DSA0120-OBJECT-ORIETED-PROGRAMMING-WITH-C-.git

Q1 – Call by Value, Reference and Pointer

1. Aim

To implement call by value, call by reference and call by pointer using a hostel fee concession example.

2. Explanation

- Call by Value: A copy of the variable is passed. Changes do not affect the original variable.
- Call by Reference: The original variable is accessed using a reference. Changes affect the original variable.
- Call by Pointer: The memory address is passed. Changes made using the pointer affect the original variable.

3. C++ Program

```
#include <iostream>

using namespace std;


void byValue(int fee, int percent)

{

   fee -= fee * percent / 100;

}


void byReference(int& fee, int percent)

{

   fee -= fee * percent / 100;

}


void byPointer(int* fee, int percent)
```

```
{
    *fee -= *fee * percent / 100;
}

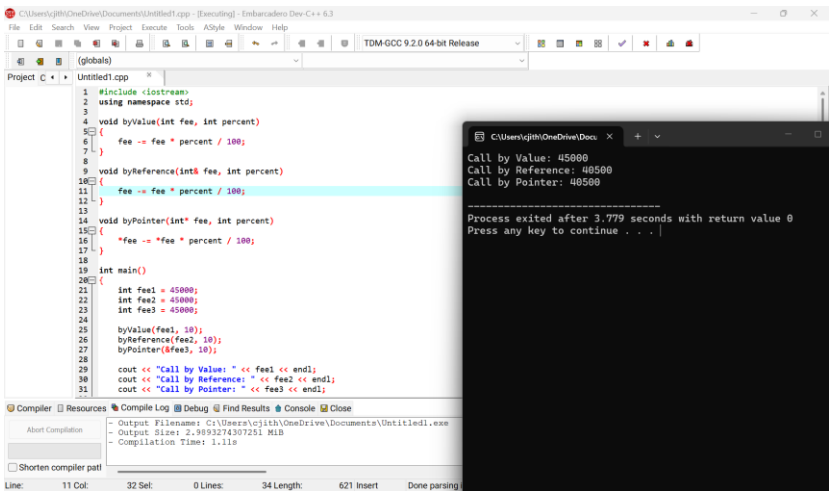
int main()
{
    int fee1 = 45000;
    int fee2 = 45000;
    int fee3 = 45000;

    byValue(fee1, 10);
    byReference(fee2, 10);
    byPointer(&fee3, 10);

    cout << "Call by Value: " << fee1 << endl;
    cout << "Call by Reference: " << fee2 << endl;
    cout << "Call by Pointer: " << fee3 << endl;

    return 0;
}
```

4. Output



5. Explanation

Call by value changes only the local copy, so the original fee remains unchanged. Call by reference and pointer modify the original fee directly.

A const reference should be used when a function needs to access a variable without modifying it.

6. Design Decision

Call by reference is preferred when multiple concessions must be applied because it directly updates the original fee without creating another variable.

7. Predict the Output

Program output:

1 12 13

1 100

16 13 6

Explanation:

- a remains 1 because it is passed by value.
- b becomes 12 because it is passed by reference.
- c becomes 13 because it is modified using a pointer.
- bigger(a,c) returns a reference to c, so c becomes 100.
- show(1) uses default arguments 5 and 10.
- show(1,2) uses default argument 10.
- show(1,2,3) uses all supplied arguments.

8. Complete the Code – TODO 1

```
#include <iostream>

using namespace std;


void applyByValue(long long fee, int percent)

{

   fee -= fee * percent / 100;

}
```

```
void applyConcession(long long& fee, int percent)
{
    fee -= fee * percent / 100;
}

int main()
{
    long long hostelFee = 45000;

    applyByValue(hostelFee, 10);

    cout << "after call by value   : " << hostelFee << '\n';

    applyConcession(hostelFee, 10);

    cout << "after call by reference: " << hostelFee << '\n';

    return 0;
}
```

Output:

```
after call by value   : 45000
after call by reference: 40500
```

9. Alternative Concept and Complexity Analysis

If the function returns the updated fee, the program becomes:

```
long long applyConcession(long long fee, int percent)
{
    return fee - fee * percent / 100;
```

}

Function call:

hostelFee = applyConcession(hostelFee, 10);

Method	Time Complexity	Extra Space
Call by Value	O(1)	O(1)
Call by Reference	O(1)	O(1)
Call by Pointer	O(1)	O(1)
Call by Const Reference	O(1)	O(1)

Passing a large Student object by value creates a complete copy, whereas reference and const reference avoid copying the object.

Q2 – Return by Reference and Its Dangers (4 Marks)

1. Aim

To demonstrate returning a reference to an array element and understand the dangers of returning a reference to a local variable.

2. C++ Program

```
#include <iostream>

using namespace std;


int& highest(int marks[], int n)

{

   int maxIndex = 0;


   for (int i = 1; i < n; i++)

   {

      if (marks[i] > marks[maxIndex])

         maxIndex = i;

   }
```

```
    return marks[maxIndex];
}

int main()
{
    int marks[5] = {75, 90, 85, 95, 80};

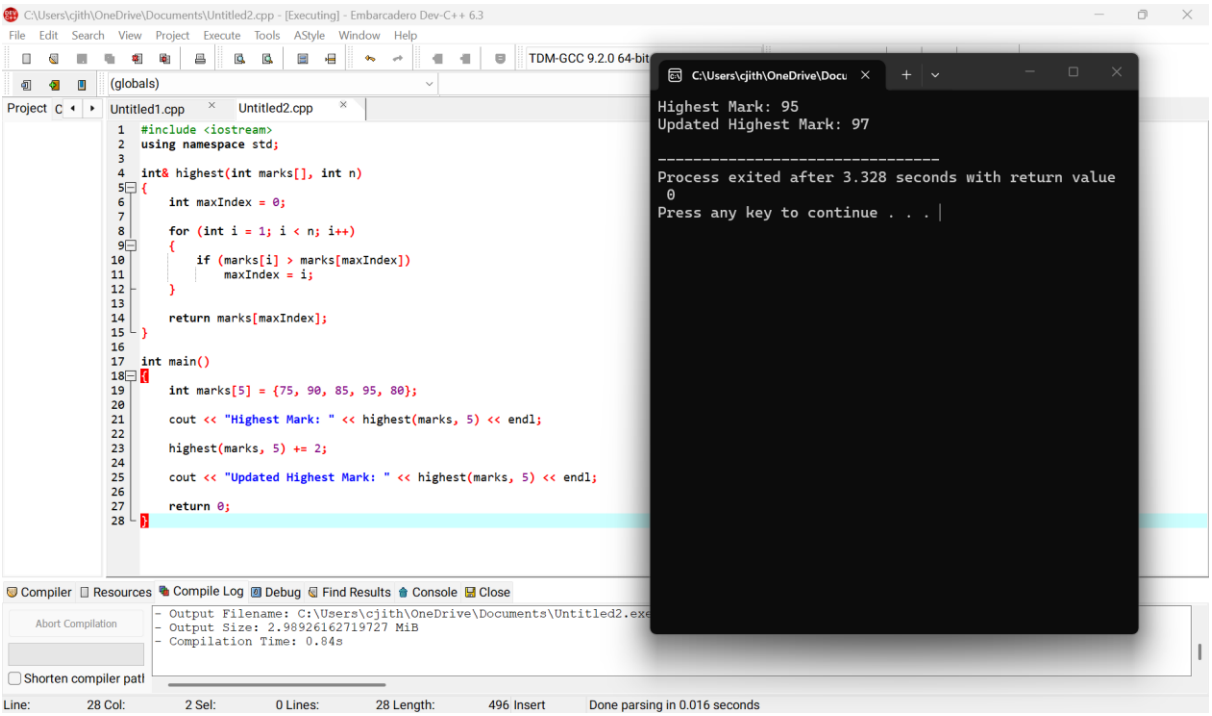
    cout << "Highest Mark: " << highest(marks, 5) << endl;

    highest(marks, 5) += 2;

    cout << "Updated Highest Mark: " << highest(marks, 5) << endl;

    return 0;
}
```

3. Output



4. Explanation

Returning a reference allows the caller to directly modify the original array element. The expression highest(marks, 5) += 2 updates the highest mark.

Returning a reference to a local variable is dangerous because the local variable is destroyed when the function ends. Using the returned reference causes undefined behaviour and the compiler may display a warning.

5. Difference Between Return Types

Return type	Description
int&	Returns a modifiable reference
const int&	Returns a reference that cannot modify the value through it
int	Returns a separate copy

6. Alternative Concept

Returning an index:

```
int highestIndex(int marks[], int n)
{
   int index = 0;

   for (int i = 1; i < n; i++)
      if (marks[i] > marks[index])
         index = i;

   return index;
}
```

Calling:

```
marks[highestIndex(marks, 5)] += 2;
```

Returning a pointer:

```
int* highestPointer(int marks[], int n)
{
   int index = 0;
```

```
    for (int i = 1; i < n; i++)
        if (marks[i] > marks[index])
            index = i;

    return &marks[index];
}
```

Calling:

```
*highestPointer(marks, 5) += 2;
```

7. Complexity Analysis

Method	Time Complexity	Extra Space
Reference	O(n)	O(1)
Index	O(n)	O(1)
Pointer	O(n)	O(1)

When a vector grows, its memory may be reallocated. Previously obtained pointers and references may become invalid. Returning an index is generally safer because the element can be accessed again using the current vector storage.

Q3 – Default Arguments and Function Overloading (6 Marks)

1. Aim

To implement function overloading and default arguments using a courier charge calculator.

2. Explanation

Function overloading allows multiple functions to have the same name with different parameter types or numbers.

Default arguments provide values automatically when optional arguments are not supplied.

The given program produces an ambiguous call because multiple overloaded functions can accept the same function call.

3. C++ Program

```
#include <iostream>

using namespace std;


struct Parcel
```

```
{
    double weight;
    bool express;
    bool insured;
};

double charge(int grams)
{
    return grams * 0.05;
}

double charge(double kg, bool express = false, bool insured = false)
{
    double total = kg * 100;

    if (express)
        total += 50;

    if (insured)
        total += 30;

    return total;
}

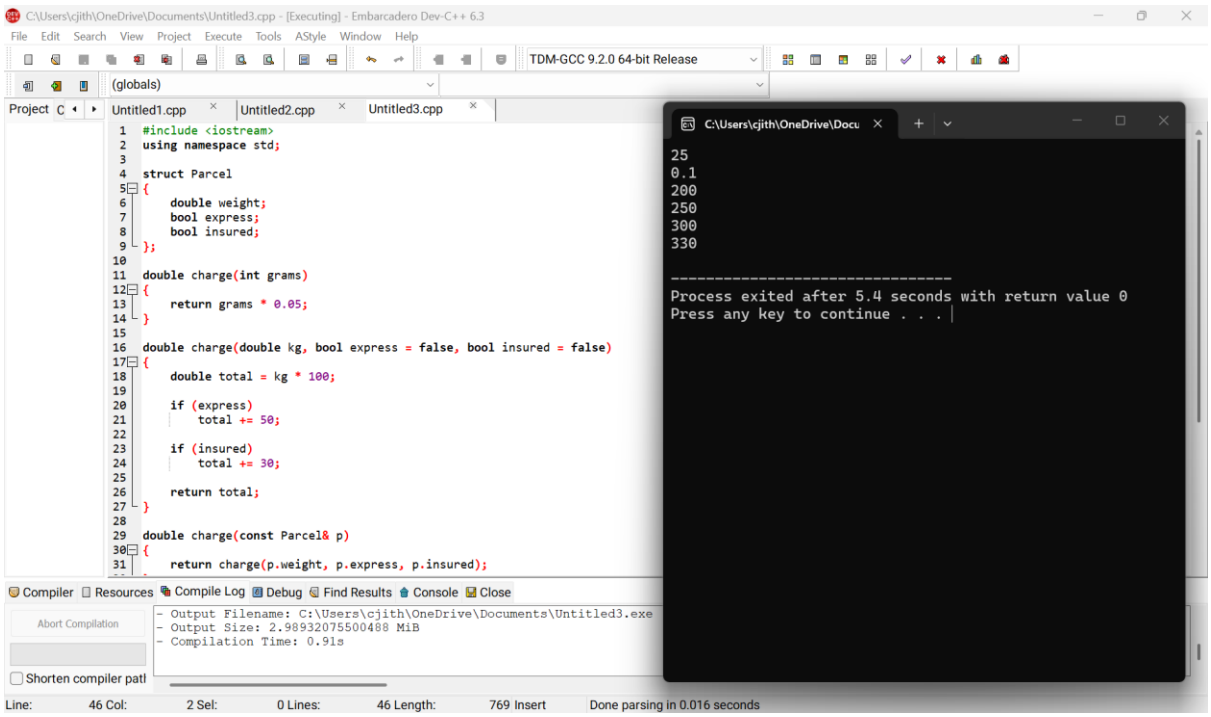
double charge(const Parcel& p)
{
    return charge(p.weight, p.express, p.insured);
}
```

```
int main()
{
    Parcel p = {2.5, true, true};

    cout << charge(500) << endl;
    cout << charge(2) << endl;
    cout << charge(2.0f) << endl;
    cout << charge(2.5) << endl;
    cout << charge(2.5, true) << endl;
    cout << charge(p) << endl;

    return 0;
}
```

4. Output



5. Explanation of Function Selection

Function call	Selected function
charge(500)	Integer overload
charge(2)	Integer overload
charge(2.0f)	Double overload
charge(2.5)	Double overload
charge(2.5, true)	Double overload

6. Overload Resolution

The compiler selects an overloaded function based on:

1. Exact match – Same parameter type.
2. Promotion – Conversion to a preferred type.
3. Conversion – Conversion from one compatible type to another.

Default arguments must be specified from right to left because trailing parameters are the ones that can be omitted.

7. Alternative Concept

A single Parcel structure can store all optional information. This makes it easier to add new options without creating many overloaded functions.

8. Complexity Analysis

- Function overload resolution happens at compile time.
- Runtime function execution takes O(1) for the calculations shown.
- Passing a Parcel by const reference avoids copying the entire object.
- Passing it by value creates an additional copy.

Concept	Time Complexity	Space
Function overloading	Compile-time selection	No runtime selection space
Default arguments	O(1) execution	O(1)
Parcel by value	O(1) for fixed-size structure	O(1)
Parcel by reference	O(1)	O(1)

Q4 – Specifying a Class with an Array Inside It

1. Aim

To create a SavingsAccount class containing an array to store the last 10 transactions.

2. C++ Program

```
#include <iostream>
using namespace std;

class SavingsAccount
{
private:
    int accountNumber;
    double balance;
    double transactions[10];
    int count;

public:
    SavingsAccount(int acc, double bal)
    {
        accountNumber = acc;
        balance = bal;
        count = 0;
    }

    void deposit(double amount);
    void withdraw(double amount);
    void miniStatement();
};

void SavingsAccount::deposit(double amount)
```

```
{
    balance += amount;

    if (count == 10)
    {
        for (int i = 1; i < 10; i++)
            transactions[i - 1] = transactions[i];

        count--;
    }

    transactions[count++] = amount;
}

void SavingsAccount::withdraw(double amount)
{
    if (amount <= balance)
    {
        balance -= amount;

        if (count == 10)
        {
            for (int i = 1; i < 10; i++)
                transactions[i - 1] = transactions[i];

            count--;
        }

        transactions[count++] = -amount;
```

```
    }
}

void SavingsAccount::miniStatement()
{
    cout << "Account Number: " << accountNumber << endl;
    cout << "Balance: " << balance << endl;
    cout << "Last Transactions:" << endl;

    for (int i = 0; i < count; i++)
        cout << transactions[i] << endl;
}

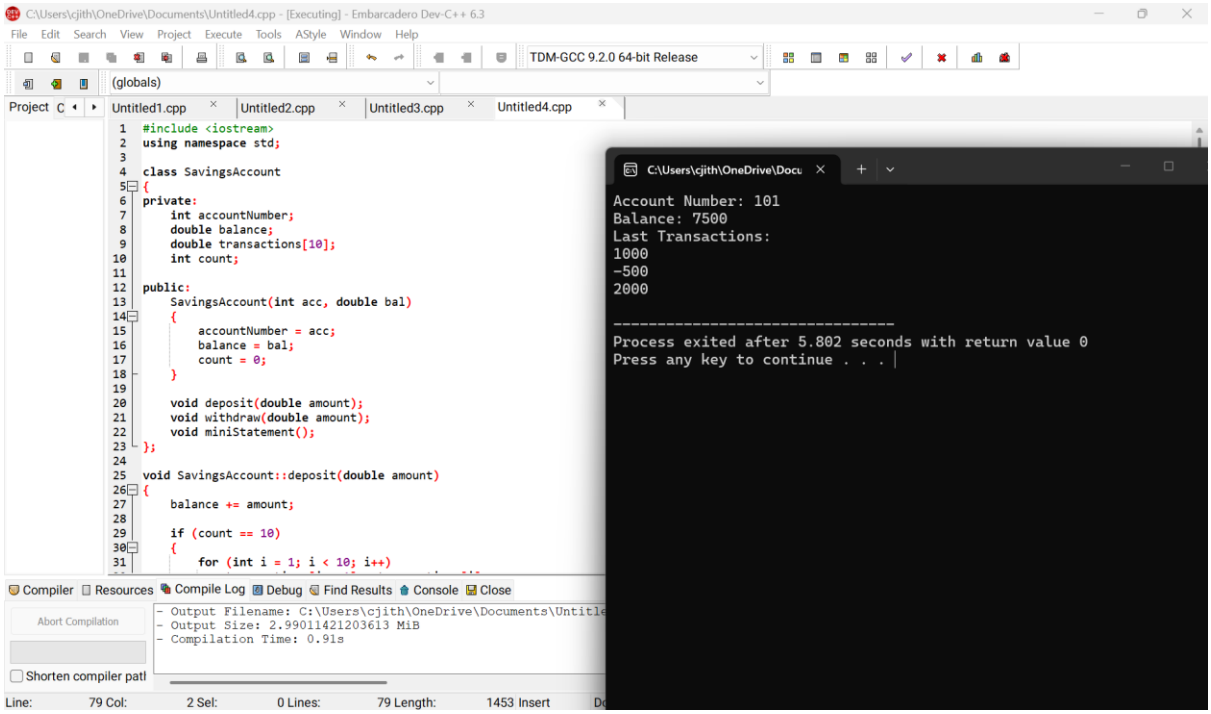
int main()
{
    SavingsAccount s(101, 5000);

    s.deposit(1000);
    s.withdraw(500);
    s.deposit(2000);

    s.miniStatement();

    return 0;
}
```

3. Output



4. Explanation

The class contains private data members such as account number, balance, transaction array and transaction count.

The array is private to protect the transaction information from direct modification outside the class.

When the 11th transaction arrives, the oldest transaction is removed and the new transaction is inserted.

5. Shifting Array vs Circular Array

Shifting Array	**Circular Array**
Moves all elements	Does not move all elements
Takes O(k) time	Takes O(1) time
Simple implementation	Requires index management
Uses fixed memory	Uses fixed memory

6. Design Decision

For storing the last 50 transactions, a circular buffer is preferred because it avoids shifting elements whenever a new transaction arrives.

7. Alternative Concept – Circular Buffer

A circular buffer uses a starting index to identify the oldest transaction.

Pseudocode:

If buffer is not full:

Insert transaction at next available position

Increase count

Else:

Replace the oldest transaction

Move start index forward circularly

8. Complexity Analysis

Method	Insertion Time	Space
Fixed array with shifting	O(k)	O(k)
Circular buffer	O(1)	O(k)
Deque	O(1) amortized	O(k)

Here, k represents the number of stored transactions.

Q5 – Static Members and Arrays of Objects (7 Marks)

1. Aim

To implement static data members and static member functions using a college bus ticket booking system.

2. Explanation

A static data member belongs to the entire class rather than an individual object. It is shared among all objects.

A static member function can access only static members directly because it does not have a this pointer.

3. Predict the Output

Given program output:

101 103 104 4

Explanation:

- The static variable next starts at 100.
- Three Token objects are created in the array.
- One additional Token object is created.

- The constructor executes four times.
- The generated IDs are 101, 102, 103 and 104.
- issued() returns 104 - 100 = 4.

4. Complete the Code – TODO 1 and TODO 2

```
#include <iostream>
#include <string>
using namespace std;

class BusTicket
{
   static int issued;
   static const int CAPACITY = 3;

   int seat;
   string name;

public:
   explicit BusTicket(const string& n) : seat(0), name(n)
   {
      if (issued < CAPACITY)
      {
         issued++;
         seat = issued;
      }
   }

   static int count()
   {
      return issued;
```

```
    }

    void print() const
    {
        cout << name << ": "
             << (seat ? "Seat " + to_string(seat) : string("Waiting"))
             << '\n';
    }
};

int BusTicket::issued = 0;

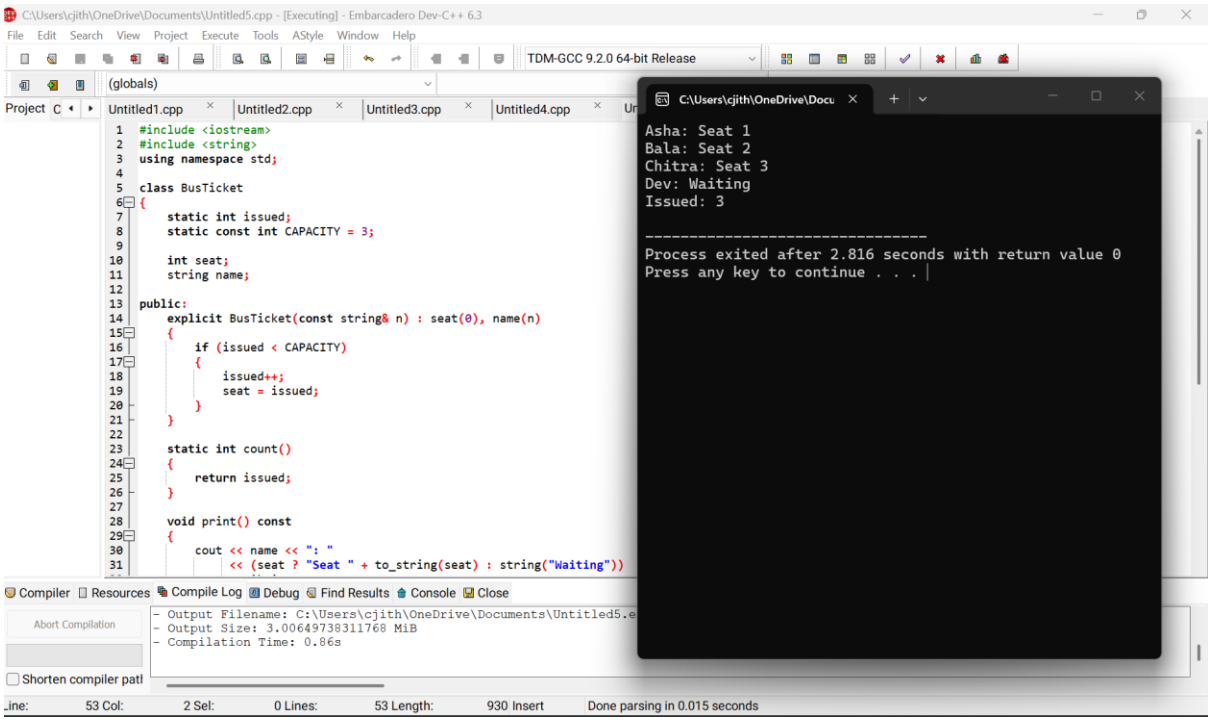
int main()
{
    BusTicket trip[4] = {
        BusTicket("Asha"),
        BusTicket("Bala"),
        BusTicket("Chitra"),
        BusTicket("Dev")
    };

    for (const BusTicket& t : trip)
        t.print();

    cout << "Issued: " << BusTicket::count() << '\n';

    return 0;
}
```

5. Output



6. Explanation

The static variable issued stores the total number of tickets issued. It is shared by every BusTicket object.

The static member is defined outside the class:

int BusTicket::issued = 0;

The array of objects creates four tickets. Only three tickets receive seat numbers because the capacity is three. The fourth ticket goes to the waiting list.

7. Creating Tickets Inside a Function

```
void createTickets()

{

   BusTicket trip[2] = {

      BusTicket("Student1"),

      BusTicket("Student2")

   };


   for (const BusTicket& t : trip)
```

```
    t.print();
}
```

If the function is called twice, the static count continues from its previous value. It does not reset when the function ends.

8. Alternative Concept

A global variable can store the ticket count, but it is accessible from different parts of the program.

A separate TicketOffice object can also manage the count. Each bus can have its own TicketOffice object, allowing multiple buses to maintain separate ticket counts.

9. Complexity Analysis

Method	Time	Space
Static member	O(1)	O(1)
Global variable	O(1)	O(1)
TicketOffice object	O(1)	O(1)

Static members provide better encapsulation because the shared count belongs to the class.

Q6 – Friend Functions and Inline Functions

1. Aim

To implement friend functions and inline functions using a Time class to calculate the difference between two times.

2. Explanation

A friend function is a non-member function that can access private and protected members of a class.

An inline function is a function for which the compiler may substitute the function body at the calling location to reduce function-call overhead. The compiler is not required to do so.

3. C++ Program – Complete the Code

```
#include <iostream>

using namespace std;


class Time

{
```

```
    int h, m;

public:
    Time(int hh, int mm) : h(hh), m(mm) {}

    friend inline int minutesBetween(const Time& a, const Time& b);

    friend inline bool earlier(const Time& a, const Time& b);
};

inline int minutesBetween(const Time& a, const Time& b)
{
    int t1 = a.h * 60 + a.m;
    int t2 = b.h * 60 + b.m;

    return (t1 > t2) ? t1 - t2 : t2 - t1;
}

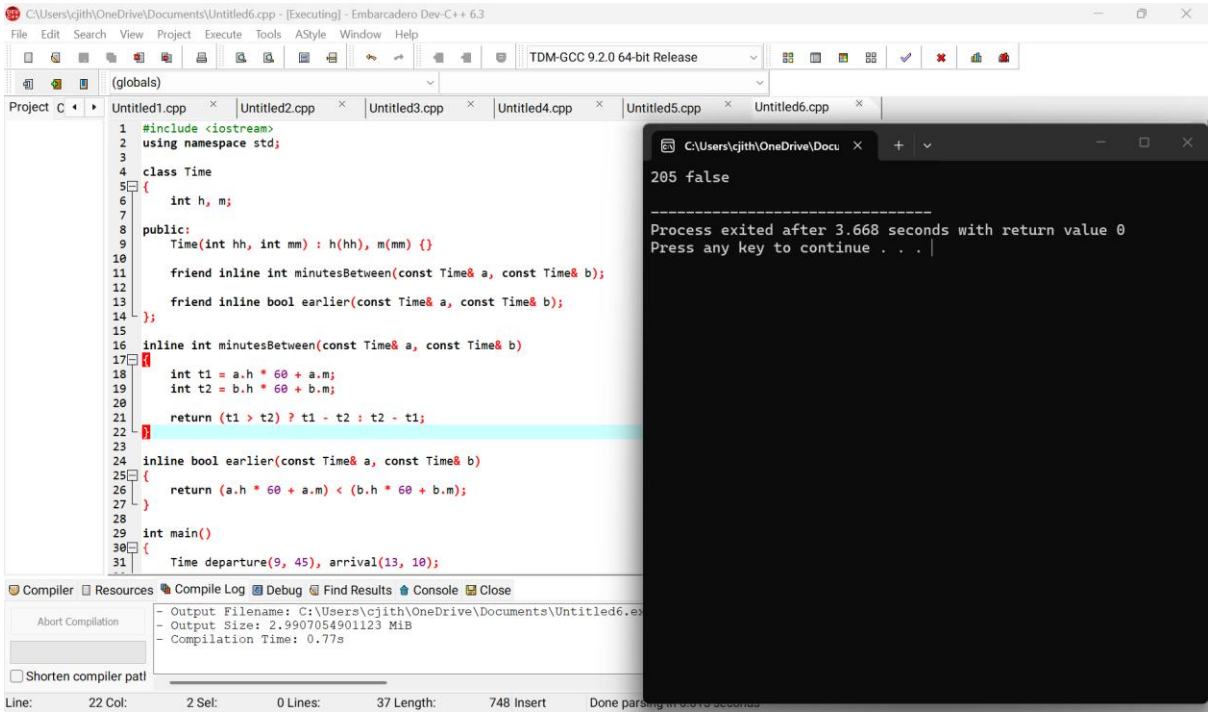
inline bool earlier(const Time& a, const Time& b)
{
    return (a.h * 60 + a.m) < (b.h * 60 + b.m);
}

int main()
{
    Time departure(9, 45), arrival(13, 10);

    cout << minutesBetween(departure, arrival) << ' '
         << boolalpha << earlier(arrival, departure) << '\n';
```

```
    return 0;
}
```

4. Output



5. Explanation

- Departure time is 9:45 AM.
- Arrival time is 1:10 PM.
- The difference between both times is 205 minutes.
- The arrival time is not earlier than the departure time, so the result is false.

6. Additional Test Cases

Time A	Time B	Difference	Earlier(A,B)
09:45	13:10	205	True
11:30	12:30	60	True
14:00	10:00	240	False
12:00	12:00	0	False

7. Friend Function vs Member Function

Friend Function	Member Function
Not a class member	Belongs to the class
Called like a normal function	Called using an object
Can access private members	Can access private members
Does not have this pointer	Has this pointer

8. Alternative Concept

The same program can be implemented using member functions:

```
class Time
{
   int h, m;

public:
   Time(int hh, int mm) : h(hh), m(mm) {}

   int minutesTo(const Time& b) const
   {
      int t1 = h * 60 + m;
      int t2 = b.h * 60 + b.m;

      return (t1 > t2) ? t1 - t2 : t2 - t1;
   }

   bool isEarlier(const Time& b) const
   {
      return (h * 60 + m) < (b.h * 60 + b.m);
   }
};
```

9. Complexity Analysis

Method	Time Complexity	Space
Friend function	O(1)	O(1)
Member function	O(1)	O(1)
Getter functions	O(1)	O(1)

Inline functions may reduce function-call overhead, but actual performance depends on compiler optimisation.

Compilation commands for performance testing:

g++ -std=c++17 -O0 main.cpp -o test

g++ -std=c++17 -O2 main.cpp -o test

Actual timing measurements must be collected by executing the program on your computer. They should not be invented.

Q7 – Diagnose, Redesign and Validate

A. Diagnose the Limitation

The original SavingsAccount program uses a fixed array to store the last 10 transactions.

When a new transaction arrives after the array becomes full, all existing elements must be shifted to the left to remove the oldest transaction.

This operation takes additional time and becomes inefficient when the number of transactions increases.

B. Define the Design Objective

The objective is to improve transaction insertion speed by replacing the shifting array with a circular buffer.

The redesigned system should:

- Store the latest 10 transactions.
- Remove the oldest transaction automatically.
- Insert new transactions in constant time.
- Use fixed memory.

C. Design the Revised Solution (4 Marks)

Revised C++ Program

#include <iostream>

```
using namespace std;

class SavingsAccount
{
    int accountNumber;
    double balance;
    double transactions[10];
    int start;
    int count;

public:
    SavingsAccount(int acc, double bal)
    {
        accountNumber = acc;
        balance = bal;
        start = 0;
        count = 0;
    }

    void deposit(double amount)
    {
        balance += amount;
        addTransaction(amount);
    }

    void withdraw(double amount)
    {
        if (amount <= balance)
        {
```

```
            balance -= amount;
            addTransaction(-amount);
        }
    }

    void addTransaction(double amount)
    {
        if (count < 10)
        {
            transactions[(start + count) % 10] = amount;
            count++;
        }
        else
        {
            transactions[start] = amount;
            start = (start + 1) % 10;
        }
    }

    void miniStatement()
    {
        cout << "Account Number: " << accountNumber << endl;
        cout << "Balance: " << balance << endl;
        cout << "Last Transactions:" << endl;

        for (int i = 0; i < count; i++)
            cout << transactions[(start + i) % 10] << endl;
    }
};
```

```
int main()
{
    SavingsAccount s(101, 5000);

    s.deposit(1000);
    s.withdraw(500);
    s.deposit(2000);
    s.deposit(3000);

    s.miniStatement();

    return 0;
}
```

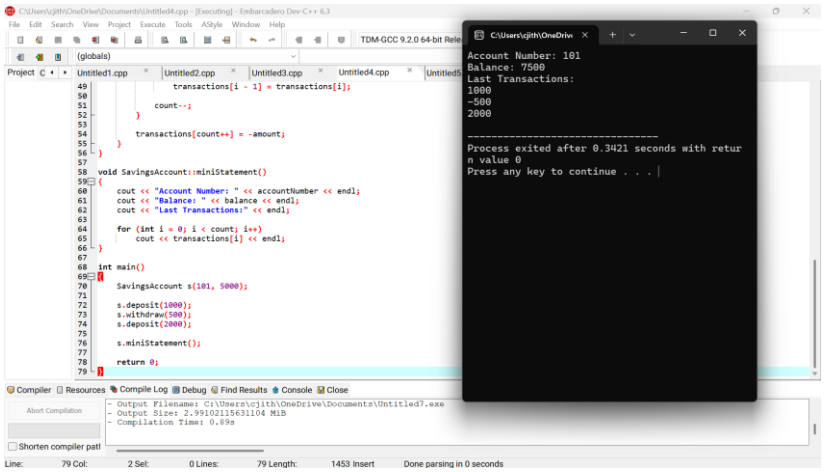
D. Predict Before Implementing

The redesigned circular buffer is expected to insert transactions faster because it does not shift existing elements.

Expected improvement:

- Original insertion: O(k)
- Redesigned insertion: O(1)
- Memory usage: O(k) in both designs.

E. Implement and Validate the Redesign



Comparison:

Feature	Original Design	Redesigned Design
Data structure	Fixed array	Circular buffer
Insertion	O(k)	O(1)
Memory	O(k)	O(k)
Element shifting	Required	Not required
Performance	Slower for full buffer	Faster insertion

F. Defend the Final Design

The circular buffer is better for applications that frequently insert new transactions because it avoids unnecessary shifting.

It also uses fixed memory and maintains only the latest 10 transactions.

However, the original array is simpler to understand and implement. Therefore, it can still be used when the number of transactions is small and performance is not important.

Conclusion:

The redesigned circular buffer improves insertion efficiency from O(k) to O(1). It is a suitable solution for managing transaction records in a banking system.